

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Sasi Kumar Megha	Reg. No.:	192471038
Section / Batch:	CS & BS	Date:	5/8/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

95
100

Code of Conduct

I, Sasikumar Megha (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Megha

RUBRICS FOR CALCULATION

Case Study

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	✓			
Application of Concepts	25%	✓			
Analysis	20%	✓			
Solution Design	20%	✓	✓		
Clarity	10%	✓			
Total Marks (100):		(95) ✓			

Faculty In-charge	Course Coordinator	Head of Department
Name: H. S. S. S. S.	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Design and Analysis of Algorithms

COURSE CODE: CSA06

COURSE OUTCOME COVERED

CO3: Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication (BL4, BL5)

Assessment Tool Weightage: 20%

QUESTIONS: 10 Total Marks:50

Annexure A

Case Study

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Case Study

Q91. Case Study: Video Streaming Metadata Sorting Engine (Merge Sort)

Problem Analysis

A video streaming platform stores a large amount of metadata (release date, popularity, language). The data must be sorted efficiently while preserving the order of duplicate values.

Issues Identified

- Large metadata collections
- Duplicate keys
- Need for stable sorting
- Scalability

Divide and Conquer Process

1. Divide the metadata into two halves.
2. Recursively sort each half.
3. Merge the sorted halves while preserving the order of equal elements.

Time Complexity

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n \log n)$

Justification

Merge Sort provides predictable performance and is **stable**, making it ideal for sorting large metadata collections with duplicate keys.

Q92. Case Study: Disaster Relief Resource Ranking System (Quick Sort)

Problem Analysis

A disaster management system must rank relief requests based on severity, urgency, and location in real time.

Issues Identified

- Pivot selection
- Uneven data distribution
- Frequent updates

- Fast response required

Divide and Conquer Process

1. Select a pivot.
2. Partition requests into smaller and larger groups.
3. Recursively sort both groups.
4. Combine the results.

Time Complexity

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n^2)$ (poor pivot choice)

Justification

Quick Sort is very fast in practice, uses little extra memory, and is suitable for dynamic real-time ranking when good pivot selection is used.

Q93. Case Study: Banking Customer Record Lookup System (Binary Search)

Problem Analysis

A bank stores customer records in sorted order and needs to retrieve account details quickly.

Requirements

- Sorted records
- Low search latency
- Reliable retrieval

Divide and Conquer Process

1. Compare the target account with the middle record.
2. If equal, return the result.
3. If smaller, search the left half.
4. If larger, search the right half.
5. Repeat until found or the range becomes empty.

Time Complexity

- Best Case: $O(1)$
- Average Case: $O(\log n)$
- Worst Case: $O(\log n)$

Justification

Binary Search greatly reduces search time and is ideal for banking systems where records are maintained in sorted order.

Q94. Case Study: Aerospace Simulation Computation Module (Matrix Multiplication)

Problem Analysis

An aerospace simulation repeatedly multiplies matrices for flight dynamics and coordinate transformations.

Challenges

- Large matrix dimensions
- Repeated calculations
- High computational cost

Divide and Conquer Process

1. Divide matrices into smaller submatrices.
2. Multiply corresponding submatrices recursively.
3. Combine the results to obtain the final matrix.

Time Complexity

- Standard Matrix Multiplication: $O(n^3)$
- Strassen's Algorithm: $O(n^{2.81})$ (approximately)

Justification

Divide-and-conquer matrix multiplication reduces computation time for large matrices and improves simulation performance.

Q95. Case Study: Enterprise Archival Sorting Strategy (Merge Sort + Quick Sort)

Problem Analysis

An enterprise must sort contracts, invoices, and communication logs under different requirements.

Issues Identified

- Stable ordering
- Memory limitations
- Different input distributions
- Large datasets

Feature	Merge	Quick Sort
---------	-------	------------

	Sort	
Stability	Stable	Not Stable
Extra Memory	$O(n)$	$O(\log n)$
Best Time	$O(n \log n)$	$O(n \log n)$
Average Time	$O(n \log n)$	$O(n \log n)$
Worst Time	$O(n \log n)$	$O(n^2)$
Large External Files	Excellent	Less Suitable

Justification

- Use Merge Sort when stable ordering and predictable performance are required.
- Use Quick Sort when memory usage is low and average-case speed is more important.

Conclusion: Merge Sort is preferred for archival records with duplicate values, while Quick Sort is suitable for fast in-memory sorting of smaller datasets.